



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



PROIECT DE DIPLOMĂ

Coordonator științific:
ș.l. dr. ing. Silviu Dumitru PAVĂL

Absolvent:
Emanuel DANALACHE

Iași, 2026



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Tehnologia Informației



Sistem de tranzacționare algoritmică pentru automatizarea și evaluarea strategiilor financiare

PROIECT DE DIPLOMĂ

Coordonator științific:
ș.l. dr. ing. Silviu Dumitru PAVĂL

Absolvent:
Emanuel DANALACHE

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII PROIECTULUI DE DIPLOMĂ

Subsemnatul DANALACHE EMANUEL,
legitimă cu CI seria XT nr. 949986 CNP 5021206070036
autorul lucrării SISTEM DE TRANZACȚIONARE ALGORITMICĂ
PENTRU AUTOMATIZAREA ȘI EVALUAREA STRATEGIILOR
FINANCIARE

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii TEHNOLOGIA INFORMAȚIEI organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea IULIE 2026 a anului universitar 2025-2026, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data
22.06.2026

Semnătura


Cuprins

1	Introducere	1
1.1	Obiectivele lucrării	1
1.2	Metodologia utilizată	2
1.3	Structura lucrării	2
2	Fundamentarea teoretică și documentarea bibliografică	3
2.1	Piața Forex și datele OHLCV	3
2.2	Indicatori tehnici utilizați	3
2.3	Regimuri de piață	4
2.4	Backtesting și riscul de supraoptimizare	4
2.5	Managementul riscului	4
2.6	Soluții similare și poziționarea proiectului	4
2.7	Cerințe derivate	5
3	Soluția propusă	6
3.1	Cerințe funcționale și nefuncționale	6
3.1.1	Cerințe ale utilizatorului	6
3.1.2	Cerințe funcționale detaliate	7
3.1.3	Cerințe nefuncționale detaliate	7
3.2	Alegerea tehnologiilor	8
3.3	Arhitectura sistemului	9
3.3.1	Responsabilitățile modulelor	9
3.3.2	Modelul datelor interne	10
3.4	Trasabilitatea cerințelor	11
3.5	Fluxul de backtesting	11
3.6	Strategiile implementate	12
3.7	Allocatorul de strategii	13
3.8	Managementul riscului	13
3.9	Modul live paper trading	14
3.10	Observabilitate în timp real	14
3.11	Artefacte generate	15
3.12	Algoritmul general	16
4	Testarea aplicației și rezultate experimentale	17
4.1	Mediul de testare	17
4.2	Punerea în funcțiune și configurarea aplicației	17
4.3	Scenarii de test	18
4.4	Metrici urmărite	18
4.5	Metodologia de testare	18
4.6	Rezultate obținute	19
4.7	Analiza riscului	19

4.8	Resurse, fiabilitate și scalabilitate	19
4.9	Aspecte de securitate și siguranță operațională	20
4.10	Testarea modului live	20
4.11	Monitorizare live cu Prometheus și Grafana	21
4.12	Capturi de ecran din testare și monitorizare	21
4.13	Limitări	23
5	Concluzii	24
5.1	Contribuții ale lucrării	24
5.2	Comparație cu soluții similare	25
5.3	Direcții viitoare de dezvoltare	25
	Bibliografie	26
	Anexe	27
1	Comenzi de rulare utilizate	27
2	Metrici de observabilitate live	27
3	Procedură de verificare pentru rularea live paper	28
4	Structura artefactelor generate	28
5	Parametri relevanți de configurare	29
6	Fragmente relevante din codul sursă	30
6.1	Modele interne pentru decizie, risc și tranzacții	30
6.2	Calculul scorului în allocatorul de strategii	30
6.3	Circuit breaker pentru controlul drawdown-ului	31
6.4	Protecție împotriva rulărilor live duplicate	32
6.5	Persistarea artefactelor de analiză	32

Sistem de tranzacționare algoritmică pentru automatizarea și evaluarea strategiilor financiare

Emanuel DANALACHE

Rezumat

Lucrarea prezintă proiectarea și implementarea unui sistem software pentru tranzacționare algoritmică pe piața Forex, orientat către testarea reproductibilă a strategiilor și către validarea controlată în regim de paper trading. Aplicația realizată, *Forex Quant Bot*, urmărește separarea clară a responsabilităților dintre colectarea datelor, generarea semnalelor, evaluarea riscului, simularea tranzacțiilor și raportarea rezultatelor. Sistemul utilizează date istorice OHLCV descărcate prin Dukascopy și păstrate local în fișiere CSV, iar pentru modul live se integrează cu Interactive Brokers printr-o conexiune limitată la conturi paper.

Contribuția principală constă în realizarea unui cadru modular care combină trei familii de strategii: urmărire de trend, revenire la medie și străpungere de canal. Semnalele individuale sunt agregate printr-un allocator care ia în calcul performanța recentă, încrederea strategiei, potrivirea cu regimul de piață și diversificarea semnalelor. Riscul este tratat ca o componentă independentă și este controlat prin dimensionarea pozițiilor în funcție de ATR, limitarea expunerii, stop-loss, take-profit, trailing stop și mecanism de oprire temporară la depășirea pragului de drawdown.

Validarea s-a realizat prin backtesting pe perechi valutare majore, folosind indicatori precum profitul net, profit factor, rata tranzacțiilor profitabile, drawdown maxim, Sharpe și Sortino. Rezultatele arată că performanța depinde semnificativ de perechea valutară și de regimul de piață, confirmând importanța evaluării sistematice înaintea utilizării operaționale.

Capitolul 1. Introducere

Piețele financiare moderne sunt caracterizate printr-un volum ridicat de date, prin actualizări frecvente ale prețurilor și printr-o complexitate tot mai mare a instrumentelor tranzacționabile. În cazul pieței Forex, aceste aspecte sunt accentuate de faptul că tranzacțiile se desfășoară aproape continuu pe durata săptămânii, iar evoluția cursurilor valutare poate fi influențată de factori macroeconomici, de lichiditate, de volatilitate și de comportamentul participanților instituționali. Într-un asemenea context, evaluarea manuală a oportunităților de tranzacționare devine dificil de realizat într-un mod constant și obiectiv.

Tranzacționarea algoritmică reprezintă o alternativă la procesul decizional exclusiv manual, deoarece permite definirea unor reguli explicite de intrare, ieșire și control al riscului. Aceste reguli pot fi testate pe date istorice, pot fi analizate prin indicatori cantitativi și pot fi executate într-un mod reproductibil. Totuși, utilizarea unui algoritm de tranzacționare nu elimină riscul financiar. Din acest motiv, o componentă esențială a oricărui astfel de sistem este capacitatea de a verifica, justifica și reproduce deciziile înainte de conectarea la un broker.

Tema acestei lucrări constă în proiectarea și implementarea unui sistem software pentru automatizarea și evaluarea strategiilor de tranzacționare pe piața Forex. În acest scop a fost dezvoltată aplicația *Forex Quant Bot*, o soluție construită pentru analiza strategiilor, simularea tranzacțiilor și verificarea controlată a comportamentului în regim live paper. Aplicația are două direcții principale de utilizare: rularea de teste istorice deterministe, pe date stocate local, și testarea operațională într-un mediu de tranzacționare simulat. Separarea acestor două moduri de funcționare este importantă, deoarece permite validarea logicii de tranzacționare într-un mediu controlat înainte de orice formă de utilizare live.

Motivația alegerii acestei teme provine din necesitatea unui instrument transparent, modular și ușor de auditat pentru evaluarea strategiilor financiare. În practică, o strategie poate părea profitabilă pe un interval restrâns, dar poate avea rezultate slabe în alt regim de piață sau poate fi afectată de supraoptimizare. Prin urmare, lucrarea nu urmărește doar obținerea unor semnale de cumpărare sau vânzare, ci construirea unui flux complet de cercetare: colectarea datelor, generarea semnalelor, agregarea deciziilor, aplicarea regulilor de risc, simularea tranzacțiilor și raportarea rezultatelor.

1.1. Obiectivele lucrării

Obiectivul general al lucrării este realizarea unui sistem modular de tranzacționare algoritmică pentru perechi valutare, capabil să ruleze strategii pe date istorice, să producă rapoarte interpretabile și să ofere o bază controlată pentru testarea în mediu paper trading.

Pentru atingerea acestui obiectiv, au fost urmărite următoarele direcții:

- proiectarea unei arhitecturi software împărțite în module distincte pentru date, strategii, alocare, risc, backtesting, rulare live și raportare;
- implementarea unui flux determinist de backtesting, în care datele sunt procesate cronologic, fără acces la informație viitoare;
- integrarea mai multor strategii complementare, adaptate pentru trend, piață laterală și episoade de volatilitate ridicată;
- agregarea semnalelor printr-un mecanism de scoring care ține cont de performanță, încredere, regim de piață și diversificare;
- aplicarea unui control al riscului bazat pe volatilitate, expunere maximă, stop-loss, take-profit și mecanism de tip circuit breaker;

- generarea de artefacte de analiză în formate accesibile, precum CSV, text, HTML și SVG;
- conectarea controlată la un broker în regim paper trading, pentru verificarea comportamentului live fără expunere financiară reală;
- expunerea de metrice operaționale pentru monitorizarea comportamentului live.

1.2. Metodologia utilizată

Metodologia de lucru a fost una incrementală. În prima etapă au fost analizate conceptele financiare necesare dezvoltării aplicației: structura datelor OHLCV, indicatorii tehnici, regimurile de piață, backtesting-ul și managementul riscului. În etapa următoare a fost definită arhitectura software, cu accent pe separarea responsabilităților, astfel încât fiecare componentă să poată fi înțeleasă, testată și extinsă independent.

Implementarea a fost realizată folosind un limbaj de programare de nivel înalt, adecvat analizei de date și prototipării rapide, împreună cu biblioteci specializate pentru prelucrarea seriilor de timp. Datele istorice sunt păstrate într-un cache local, pentru a evita descărcările repetate și pentru a asigura reproductibilitatea testelor. În modul de backtesting, motorul parcurge datele bară cu bară, calculează indicatorii, detectează regimul de piață, evaluează strategiile, aplică regulile de risc și înregistrează tranzacțiile rezultate.

Pentru modul live, sistemul se conectează controlat la un broker în regim paper trading, folosind exclusiv mecanisme asociate conturilor simulate. Această restricție este o măsură deliberată de siguranță, deoarece proiectul are scop educațional și experimental, nu scop de execuție financiară pe cont real. În plus, componenta de observabilitate expune metrice operaționale și permite urmărirea separată a perechilor valutare monitorizate.

1.3. Structura lucrării

Lucrarea este organizată în cinci capitole. Capitolul 2 prezintă fundamentarea teoretică și documentarea bibliografică, cu accent pe tranzacționarea algoritmică, piața Forex, indicatorii tehnici, backtesting și managementul riscului. Capitolul 3 descrie soluția propusă, cerințele, arhitectura aplicației, modulele implementate, strategiile, allocatorul de semnale și componenta de risc. Capitolul 4 prezintă mediul de testare, rezultatele experimentale, analiza riscului și monitorizarea live. Capitolul 5 sintetizează concluziile, contribuțiile lucrării, limitările proiectului și direcțiile posibile de dezvoltare.

Capitolul 2. Fundamentarea teoretică și documentarea bibliografică

Tranzacționarea algoritmică poate fi definită ca procesul prin care deciziile de cumpărare, vânzare sau menținere a unei poziții sunt generate automat, pe baza unor reguli formale. Spre deosebire de tranzacționarea discreționară, în care decizia depinde în mare măsură de interpretarea subiectivă a operatorului, un sistem algoritmic aplică aceeași logică asupra fiecărei observații disponibile. Această proprietate este importantă deoarece permite reproducerea rezultatelor și analiza obiectivă a comportamentului strategiei.

În literatura de specialitate se subliniază faptul că o strategie de tranzacționare nu trebuie evaluată doar prin exemple izolate de tranzacții profitabile, ci printr-un proces riguros de testare, măsurare a riscului și verificare a robusteții [1]. În acest sens, prezenta lucrare tratează backtesting-ul, managementul riscului și raportarea rezultatelor ca elemente centrale, nu ca funcționalități secundare.

2.1. Piața Forex și datele OHLCV

Piața Forex reprezintă piața globală de schimb valutar, în care instrumentele sunt exprimate sub forma unor perechi valutare, precum EURUSD, GBPUSD sau USDJPY. Prima monedă din pereche poartă denumirea de monedă de bază, iar a doua monedă este moneda cotate. Prețul perechii indică numărul de unități din moneda cotate necesare pentru a cumpăra o unitate din moneda de bază. Această structură influențează modul de calcul al profitului și pierderii, mai ales atunci când moneda cotate nu este USD.

Datele utilizate în proiect sunt de tip OHLCV, adică includ prețul de deschidere, maximum, minimum, prețul de închidere și volumul pentru un interval de timp. Sistemul suportă mai multe granularități, de la intervale intraday până la timeframe-uri zilnice sau săptămânale. În contextul backtesting-ului, ordinea cronologică a acestor observații este esențială. Un algoritm care ar utiliza informații din viitor ar produce rezultate nerealiste, motiv pentru care toate componentele proiectului calculează indicatorii folosind doar date disponibile până la momentul curent.

2.2. Indicatori tehnici utilizați

Strategiile implementate se bazează pe indicatori tehnici clasici, aleși pentru interpretabilitate și pentru compatibilitatea lor cu datele OHLCV. Scopul acestor indicatori nu este de a prezice perfect evoluția prețului, ci de a descrie anumite caracteristici ale pieței, precum direcția, volatilitatea sau poziționarea prețului față de o medie.

Media mobilă exponențială (EMA) acordă o importanță mai mare observațiilor recente și este utilizată pentru identificarea direcției trendului. În strategia de trend, o EMA rapidă este comparată cu o EMA lentă, iar semnalul este confirmat suplimentar prin MACD și ADX.

Relative Strength Index (RSI) măsoară raportul dintre mișcările recente ascendente și descendente. În cadrul strategiei de revenire la medie, valorile foarte mici ale RSI pot indica o zonă de supravânzare, iar valorile foarte mari pot indica supracumpărare.

Benzile Bollinger definesc un canal statistic în jurul unei medii mobile. Apropierea prețului de banda inferioară sau de banda superioară poate semnala o abatere temporară față de nivelul mediu, aspect util în regimuri de piață laterale.

Canalele Donchian utilizează maximele și minimele ultimelor bare pentru identificarea străpungerilor. Într-un regim de trend sau de volatilitate ridicată, depășirea canalului poate indica o posibilă continuare a mișcării.

Average True Range (ATR) estimează volatilitatea și este folosit atât pentru validarea semnalelor, cât și pentru dimensionarea stop-loss-ului și a poziției. Prin raportarea riscului la ATR, sistemul adaptează dimensiunea tranzacției la condițiile curente de piață.

Average Directional Index (ADX) este utilizat pentru evaluarea forței trendului, iar exponentul

Hurst oferă o perspectivă suplimentară asupra caracterului persistent sau oscilant al seriei de preț.

2.3. Regimuri de piață

Un sistem de tranzacționare care aplică aceeași logică în toate condițiile de piață poate avea performanțe instabile. Piața poate evolua direcțional, lateral sau poate traversa perioade de volatilitate ridicată. Din acest motiv, proiectul clasifică fiecare bară într-un regim de piață: *trend_up*, *trend_down*, *range* sau *volatility_spike*. Clasificarea se bazează pe indicatori precum ADX, ATR, z-score-ul volatilității, exponentul Hurst și poziția prețului față de o medie mobilă.

Această clasificare permite activarea strategiilor în condițiile pentru care au fost proiectate. Strategia de trend este relevantă în regimuri direcționale, strategia de revenire la medie este mai potrivită pentru piețe laterale, iar strategia de breakout devine utilă atunci când volatilitatea crește sau când prețul depășește intervale recente importante.

2.4. Backtesting și riscul de supraoptimizare

Backtesting-ul reprezintă simularea aplicării unei strategii pe date istorice. Acesta este un pas necesar în evaluarea inițială a unei strategii, dar nu constituie o garanție pentru performanța viitoare. O problemă frecventă este supraoptimizarea, adică ajustarea excesivă a parametrilor pentru un set istoric particular. Într-o astfel de situație, strategia poate obține rezultate bune pe perioada testată, dar poate eșua atunci când este aplicată pe date noi. Bailey et al. evidențiază necesitatea unei evaluări prudente, mai ales atunci când sunt testate multe variante ale aceleiași strategii [2].

În cadrul proiectului, backtesting-ul este construit determinist. Datele sunt încărcate din cache local, fiecare bară este procesată o singură dată, iar rezultatele sunt salvate într-un director dedicat. Această abordare permite reluarea experimentelor și verificarea tranzacțiilor generate, aspect important pentru o lucrare tehnică în care rezultatele trebuie să poată fi justificate.

2.5. Managementul riscului

Managementul riscului are un rol central în sistemele de tranzacționare, deoarece rezultatul unei strategii depinde nu doar de calitatea semnalelor, ci și de modul în care sunt controlate expunerea, volatilitatea și pierderile succesive. O strategie poate genera semnale corecte din punct de vedere tehnic, dar poate produce pierderi semnificative dacă dimensiunea pozițiilor nu este controlată. În proiect, fiecare intrare este validată de o componentă de risc independentă, care ține cont de capitalul disponibil, volatilitate, expunere totală și drawdown.

Regula implicită presupune riscul a 1% din capital pentru o tranzacție, iar expunerea totală este limitată. Stop-loss-ul este calculat ca multiplu de ATR, iar take-profit-ul utilizează o distanță adaptată volatilității. Sistemul include, de asemenea, mecanisme de protecție precum break-even, trailing stop și circuit breaker. Dacă drawdown-ul depășește pragul configurat, tranzacționarea este dezactivată temporar, pentru a preveni acumularea rapidă a pierderilor.

2.6. Soluții similare și poziționarea proiectului

Există numeroase soluții pentru tranzacționare algoritmică, de la platforme comerciale consacrate până la framework-uri open-source pentru cercetare cantitativă. Platformele integrate, precum MetaTrader, oferă conectare directă la broker și un ecosistem matur, însă pot introduce dependențe de platformă, de limbajul specific și de calitatea infrastructurii brokerului. Framework-urile Python, precum Backtrader sau Zipline, sunt flexibile și potrivite pentru cercetare, dar necesită adesea adaptări pentru integrare live, raportare și gestionarea particularităților pieței Forex. Platformele cloud, precum QuantConnect, reduc efortul operațional, dar introduc dependențe externe și costuri suplimentare.

Soluția propusă se poziționează ca un proiect educațional și experimental, orientat către claritatea fluxului decizional. Sistemul nu urmărește execuție de frecvență înaltă și nu promite profitabilitate, ci oferă un cadru prin care strategiile pot fi evaluate, comparate și documentate. Valoarea

Tabelul 2.1. Comparație între tipuri de soluții pentru tranzacționare algoritmică

Soluție	Avantaje	Limitări	Poziționarea proiectului
MetaTrader / roboți Expert Advisor	Integrare directă cu brokeri, interfață matură, ecosistem larg.	Dependență de platformă, limbaj specific, control mai redus asupra fluxului de cercetare.	Proiectul oferă cod Python transparent și ușor de auditat.
Framework-uri Python de backtesting	Flexibilitate, biblioteci numerice mature, potrivite pentru cercetare.	Necesită integrare separată cu brokeri și raportare personalizată.	Proiectul include atât backtesting, cât și integrare paper cu Interactive Brokers.
Platforme cloud de trading algoritmic	Infrastructură administrată, date și execuție integrate.	Costuri, dependență de furnizor și control mai redus asupra mediului local.	Proiectul rulează local, cu date cacheuite și rezultate reproductibile.
Scripturi simple de semnalizare	Ușor de implementat și modificat.	Lipsă de arhitectură, risc slab controlat, raportare incompletă.	Proiectul separă datele, strategiile, riscul, execuția și raportarea.

principală a proiectului constă în faptul că fiecare etapă a deciziei poate fi urmărită: de la datele de intrare, până la semnal, validarea riscului și rezultatul final al tranzacției.

Lacuna pe care proiectul o acoperă este lipsa unei soluții compacte, ușor de inspectat, care să combine într-un singur cod sursă descărcarea și cache-ul datelor Forex, backtesting determinist, strategii complementare, allocator explicabil, management de risc independent, integrare paper trading și export de artefacte pentru analiză.

2.7. Cerințe derivate

Din analiza domeniului rezultă următoarele cerințe pentru aplicație:

- datele istorice trebuie să fie curate, ordonate cronologic și reutilizabile;
- strategiile trebuie să fie independente și ușor de extins;
- decizia finală trebuie să fie explicabilă prin scorurile strategiilor participante;
- riscul trebuie aplicat înainte de deschiderea fiecărei poziții;
- rezultatele trebuie salvate în formate care permit auditarea tranzacțiilor;
- modul live trebuie să includă protecții împotriva rulării accidentale pe conturi reale;
- monitorizarea live trebuie să acopere prețurile, semnalele, pozițiile și performanța curentă.

Prin realizarea acestei analize comparative și a documentării bibliografice, a fost atins primul obiectiv al lucrării, respectiv studiul literaturii de specialitate și identificarea cerințelor relevante pentru un sistem de tranzacționare algoritmică. Analiza confirmă necesitatea dezvoltării unei soluții integrate, reproductibile și ușor de auditat, care să combine backtesting-ul determinist, controlul riscului, raportarea rezultatelor și verificarea controlată în regim paper trading.

Capitolul 3. Soluția propusă

Soluția propusă este o aplicație Python modulară pentru tranzacționare algoritmică pe piața Forex. Proiectul a fost conceput astfel încât aceeași logică decizională să poată fi utilizată atât în testarea istorică, cât și în rularea live controlată. Cele două moduri principale de funcționare sunt *backtest*, pentru simularea strategiilor pe date istorice, și *live*, pentru paper trading prin Interactive Brokers.

În ambele cazuri, deciziile sunt generate printr-un flux comun: datele sunt pregătite, este identificat regimul de piață, strategiile produc semnale, allocatorul agregă aceste semnale, iar componenta de risc stabilește dacă poziția propusă poate fi deschisă. Această organizare reduce duplicarea logicii și permite compararea mai coerentă între rezultatele obținute în backtesting și comportamentul observat în rularea live.

3.1. Cerințe funcționale și nefuncționale

Aplicația trebuie să permită rularea unui backtest pentru o pereche valutară, un interval de timp, o dată de început și o dată de final. Utilizatorul poate configura capitalul inițial, riscul pe tranzacție, slippage-ul, directorul de cache și directorul de ieșire. Din punct de vedere funcțional, sistemul trebuie să poată descărca sau reutiliza date istorice, să genereze tranzacții simulate și să salveze rezultatele într-o formă care poate fi verificată ulterior.

Pentru modul live, aplicația trebuie să se conecteze la TWS sau IB Gateway în regim paper trading, să monitorizeze perechile selectate și să trimită ordine de piață doar dacă toate filtrele sunt îndeplinite. O cerință importantă este prevenirea utilizării accidentale a unui cont real; de aceea configurația implicită acceptă doar porturile paper 7497 și 4002.

Cerințele nefuncționale urmăresc reproductibilitatea, modularitatea și trasabilitatea. Backtest-ul trebuie să poată fi reluat cu aceiași parametri, iar rezultatele trebuie să includă fișiere separate pentru tranzacții, curba capitalului, metrici de risc, metrici de strategie și sumar text. Aceste cerințe au fost formulate pentru ca aplicația să poată fi analizată nu doar ca prototip funcțional, ci și ca sistem tehnic documentabil.

3.1.1. Cerințe ale utilizatorului

Utilizatorul țintă al aplicației este o persoană care dorește să testeze strategii Forex în mod reproductibil, fără a depinde de o interfață grafică complexă și fără a trimite ordine reale înainte de validare. Din această perspectivă, cerințele utilizatorului sunt:

- rularea rapidă a unui backtest prin linia de comandă, cu parametri expliți;
- selectarea perechii valutare, a timeframe-ului, a perioadei istorice și a capitalului inițial;
- reutilizarea datelor istorice locale pentru a evita descărcări repetate;
- obținerea unui sumar ușor de citit după fiecare rulare;
- inspectarea tranzacțiilor individuale, a motivelor de intrare și a motivelor de ieșire;
- testarea modului live exclusiv pe cont paper Interactive Brokers;
- păstrarea rezultatelor în fișiere standard, care pot fi analizate ulterior în Excel, Python sau alte instrumente.

3.1.2. Cerințe funcționale detaliate

Tabelul 3.1 detaliază funcționalitățile care trebuie asigurate de aplicație pentru ca fluxul de backtesting și rularea live paper să poată fi utilizate într-un mod coerent. Cerințele au fost formulate pornind de la operațiile principale ale utilizatorului: selectarea datelor, generarea semnalelor, aplicarea regulilor de risc, conectarea la broker și obținerea artefactelor de analiză.

Tabelul 3.1. Cerințe funcționale ale sistemului

Cod	Descriere	Prioritate
CF1	Sistemul permite rularea unui backtest pentru o pereche Forex, un timeframe și un interval temporal definite de utilizator, astfel încât scenariile experimentale să poată fi reluate cu aceiași parametri.	Ridică
CF2	Sistemul încarcă date istorice din cache local sau le descarcă automat atunci când lipsesc, reducând dependența de descărcări repetate și menținând continuitatea procesului de testare.	Ridică
CF3	Sistemul calculează indicatorii tehnici necesari strategiilor și detectorului de regim, pentru ca deciziile să fie generate pe baza acelorași transformări ale seriilor OHLCV.	Ridică
CF4	Sistemul combină semnalele strategiilor într-o decizie finală explicabilă, printr-un mecanism de alocare care păstrează contribuția fiecărei strategii.	Ridică
CF5	Sistemul verifică fiecare intrare prin reguli de risc înainte de deschiderea poziției, incluzând validarea semnalului, dimensiunea poziției și limitele de expunere.	Ridică
CF6	Sistemul salvează rapoarte CSV, sumar text și dashboard-uri după fiecare rulare, pentru ca rezultatele să poată fi inspectate, arhivate și comparate ulterior.	Medie
CF7	Sistemul permite conectarea la Interactive Brokers doar în regim paper trading, prevenind utilizarea accidentală a unui cont real în etapa de testare.	Ridică
CF8	Sistemul expune metrice live pentru monitorizare prin Prometheus și vizualizare în Grafana, astfel încât prețurile, semnalele și starea pozițiilor să poată fi urmărite în timp real.	Medie

3.1.3. Cerințe nefuncționale detaliate

Cerințele nefuncționale stabilesc proprietățile de calitate urmărite în implementare. În cazul unei aplicații de tranzacționare algoritmică, aceste cerințe sunt importante deoarece sistemul trebuie să fie reproductibil, ușor de auditat și suficient de sigur pentru a evita execuții nedorite în medii reale.

Tabelul 3.2. Cerințe nefuncționale ale sistemului

Cod	Descriere	Categorie
CNF1	Rulările de backtesting trebuie să fie reproductibile pentru aceleași date și aceiași parametri, pentru ca rezultatele să poată fi verificate independent.	Reproductibilitate

CNF2	Modulele trebuie să poată fi extinse fără modificarea întregului sistem, astfel încât o strategie nouă sau o regulă de risc nouă să poată fi integrată controlat.	Mentenabilitate
CNF3	Deciziile trebuie să poată fi auditate prin fișierele generate, incluzând tranzacțiile, motivele de intrare sau ieșire și evoluția capitalului.	Trasabilitate
CNF4	Aplicația trebuie să prevină conectarea accidentală la porturi non-paper Interactive Brokers, pentru a limita riscul operațional în etapa de testare.	Siguranță operațională
CNF5	Erorile de conexiune live trebuie tratate prin mesaje clare și reconectare controlată, astfel încât utilizatorul să poată identifica rapid cauza întreruperii.	Fiabilitate
CNF6	Serviciile auxiliare de monitorizare trebuie să poată rula pe porturi non-default pentru evitarea conflictelor locale cu alte instanțe Prometheus sau Grafana.	Operabilitate

3.2. Alegerea tehnologiilor

Implementarea a fost realizată în Python deoarece limbajul are un ecosistem matur pentru analiză de date, integrare cu API-uri externe și prototipare rapidă. Bibliotecile pandas [3] și NumPy [4] sunt utilizate pentru prelucrarea seriilor de timp, calculul indicatorilor și agregarea rezultatelor. PyYAML permite separarea parametrilor de configurare de codul sursă, iar dukascopy-python asigură accesul la date istorice Forex [5]. Pentru modul live, ib_insync oferă o interfață Python peste API-ul Interactive Brokers [6, 7]. Monitorizarea live utilizează prometheus-client, Prometheus, Grafana și Docker Compose pentru colectarea, stocarea și vizualizarea metricilor operaționale [8, 9, 10].

Alegerea unei aplicații de tip CLI, în locul unei interfețe grafice, este justificată de caracterul experimental al proiectului. Parametrii pot fi salvați în comenzi reproductibile, rulările pot fi automatizate, iar rezultatele sunt persistate în fișiere.

Tabelul 3.3. Tehnologii utilizate și rolul acestora

Tehnologie	Rol în proiect	Justificare
Python	Limbaj principal de implementare	Ecosistem puternic pentru date financiare și prototipare.
pandas / NumPy	Prelucrare date, indicatori, tabele de rezultate	Performanță suficientă pentru backtesting pe date OHLCV și API familiar.
Dukascopy	Sursă de date istorice Forex	Date accesibile și potrivite pentru cache local.
Interactive Brokers	Broker pentru paper trading	Permite testare operațională fără expunere financiară reală.
Prometheus / Grafana	Observabilitate live	Colectează metrice expuse de bot și le afișează în dashboard-uri per paritate.
Docker Compose	Orchestrare locală monitorizare	Pornește Prometheus și Grafana izolat de mediul Python al botului.
CSV / HTML / SVG	Formate de raportare	Ușor de inspectat, arhivat și analizat ulterior.

3.3. Arhitectura sistemului

Figura 3.1 prezintă arhitectura generală a sistemului. Aplicația este organizată în module cu responsabilități clare: interfață CLI, configurare, date, strategii, nucleu decizional, backtesting, live trading și raportare.

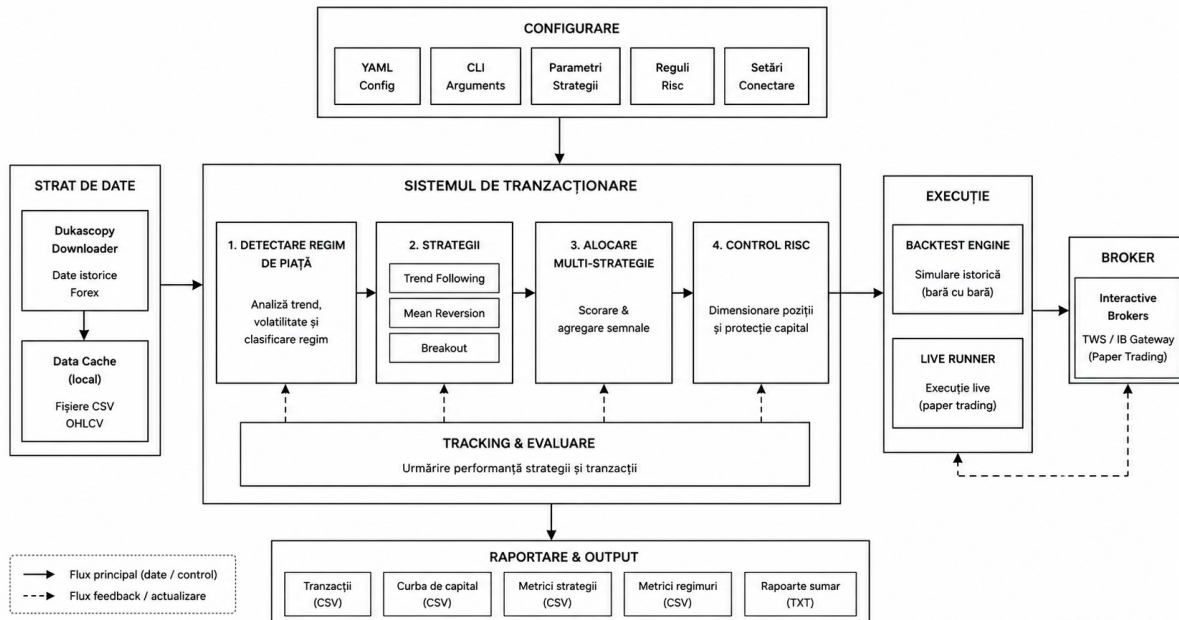


Figura 3.1. Arhitectura sistemului de tranzacționare algoritmică

În directorul `forex_quant_bot`, componentele principale sunt:

- `backtest`: motorul de simulare și calculul rapoartelor;
- `core`: detectorul de regim, allocatorul, componenta de risc și tracker-ul de performanță;
- `data`: descărcarea, cache-ul și replay-ul cronologic al datelor;
- `live`: integrarea cu Interactive Brokers și orchestrarea modului live;
- `logs`: persistarea artefactelor CSV, HTML, SVG și text;
- `strategies`: strategiile de trend, mean reversion și breakout;
- `settings.py`: clasele de configurare și suprascrierile per pereche valutară;
- `cli.py`: parsarea parametrilor din linia de comandă și selectarea modului de rulare.

3.3.1. Responsabilitățile modulelor

Datele OHLCV sunt obținute și normalizate în modulul `data`. Clasa `DataCache` verifică fișierele locale, validează acoperirea temporală și declanșează descărcarea datelor atunci când este necesar. Această separare este importantă deoarece motorul de backtesting primește doar un set de date ordonat cronologic, fără să depindă de detaliile furnizorului extern.

Strategiile propriu-zise sunt grupate în modulul `strategies`. Fiecare strategie pregătește indicatorii necesari și evaluează istoricul disponibil la momentul curent. Rezultatul este un obiect `StrategyDecision`, care include semnalul, încrederea și metadatele explicative. Această structură face posibilă adăugarea unei noi strategii fără modificarea motorului de simulare.

Modulul `core` grupează componentele comune ale deciziei. Detectorul de regim adaugă contextul de piață, allocatorul stabilește ponderea semnalelor, tracker-ul calculează performanța recentă, iar componenta de risc verifică dacă semnalul poate fi transformat în poziție.

Modulul `backtest` orchestrează simularea. El parcurge barele istorice în ordine cronologică, verifică pozițiile deschise, evaluează strategiile, aplică riscul și salvează tranzacțiile. Modulul `live` reutilizează aceeași logică decizională, dar înlocuiește sursa de date cu fluxuri Interactive Brokers și transformă intrările/ieșirile în ordine `paper`.

3.3.2. Modelul datelor interne

Datele și deciziile importante sunt reprezentate prin clase de tip `dataclass`. Această alegere oferă o structură clară, cu atribute explicite, fără complexitatea unei ierarhii de clase inutile. Printre modelele principale se află:

- `StrategyDecision`: semnalul unei strategii, încrederea și metadatele aferente;
- `CompositeDecision`: semnalul final rezultat din allocator;
- `RiskDecision`: aprobarea sau respingerea unei intrări și motivul deciziei;
- `Position`: starea unei poziții deschise;
- `TradeRecord`: tranzacția închisă, folosită pentru metrice;
- `TradeEventRecord`: evenimentele individuale de intrare și ieșire.

Prin aceste structuri, sistemul păstrează o separare clară între semnal, decizie de risc, poziție și rezultat final. Această separare ajută la auditarea rulărilor: dacă o tranzacție este respinsă, motivul se regăsește în decizia de risc; dacă o tranzacție este deschisă, motivul semnalului se păstrează în înregistrarea poziției și apoi în fișierul `trades.csv`.

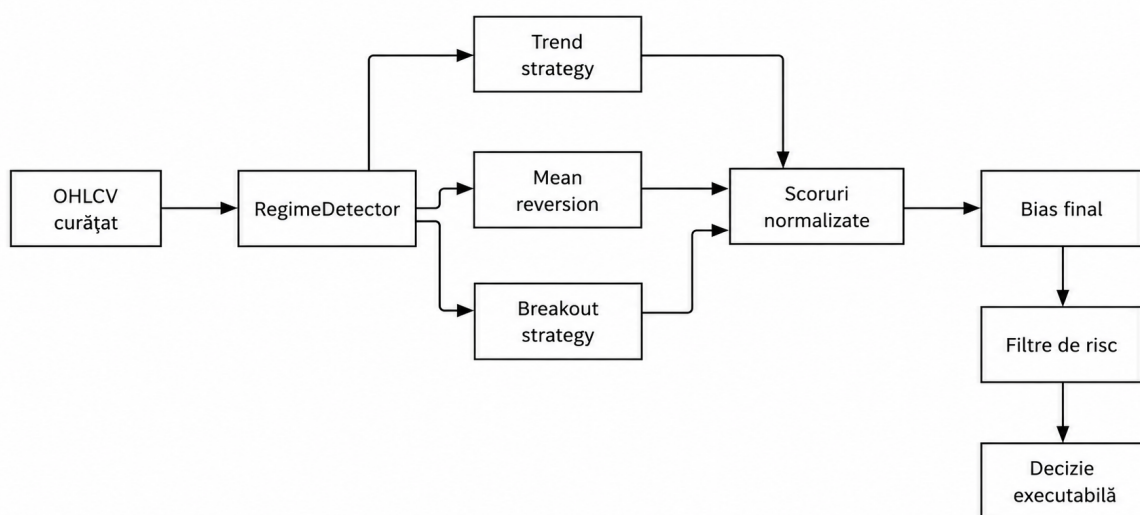


Figura 3.2. Pipeline-ul decizional al strategiilor

3.4. Trasabilitatea cerințelor

Tabelul 3.4 leagă cerințele principale de modulele responsabile și de artefactele prin care implementarea poate fi verificată.

Tabelul 3.4. Trasabilitatea cerințelor către modulele implementate

Cerință	Componentă responsabilă	Dovadă în proiect
Backtesting reproductibil	BacktestEngine, BarReplay, DataCache	Rulări deterministe pe CSV și directoare timestamped în output.
Strategii extensibile	BaseStrategy și modulele din strategies	Fiecare strategie implementează prepare_data și evaluate.
Decizie explicabilă	StrategyAllocator, StrategyDecision	Scoruri pe strategie, meta-date și motivul semnalului în CSV-uri.
Controlul riscului	RiskOverlay, RiskConfig	Stop-uri ATR, sizing, expunere maximă și circuit breaker.
Integrare broker	IBPaperBroker, LiveRunner	Validare porturi paper 7497/4002 și lock-uri pe pereche.
Raportare completă	CSVLogger, SummaryPrinter, performance_dashboard	Fișiere CSV, sumar text, HTML și SVG după rulare.

3.5. Fluxul de backtesting

În modul backtest, utilizatorul pornește aplicația printr-o comandă de forma:

```
1 python main.py --mode backtest --pair EURUSD --start 2018-01-01 \
2   --end 2024-12-31 --timeframe D
```

Cod 3.1. Exemplu de comandă pentru rularea unui backtest

Motorul verifică existența datelor în directorul data_cache. Dacă acoperirea temporală este suficientă, fișierul CSV local este reutilizat. În caz contrar, datele sunt descărcate prin Dukascopy, dacă descărcarea automată este activă. Datele sunt apoi adnotate cu indicatori de regim și cu indicatorii specifici fiecărei strategii.

Procesarea se face bară cu bară. Pentru fiecare moment, motorul actualizează poziția curentă, verifică stop-loss-ul și take-profit-ul, evaluează strategiile și transmite semnalul compus către componenta de risc. Dacă riscul este aprobat, este creată o poziție nouă. Dacă există deja o poziție, aceasta poate fi închisă prin stop, take-profit, trailing stop, break-even sau semnal opus.

Fragmentul următor sintetizează fluxul de decizie folosit în backtesting. El evidențiază faptul că strategiile sunt evaluate pe istoricul disponibil la momentul curent, iar decizia compusă este transmisă separat către componenta de risc.

```
1 for event in BarReplay(market_data, warmup_bars=warmup_bars):
2     bar = event.bar
3     close_price = float(bar["close"])
4
5     decisions = {
6         strategy.name: strategy.evaluate(event.history)
7         for strategy in strategies
```

```

8     }
9     composite = allocator.allocate(
10         decisions,
11         current_regime=str(bar["regime"]),
12         performance_tracker=tracker,
13     )
14
15     risk_decision = risk_overlay.evaluate_entry(
16         pair=pair,
17         open_positions=[],
18         bias=composite.bias,
19         equity=equity,
20         current_open_risk_ratio=open_risk_ratio,
21         close_price=close_price,
22         atr_value=float(bar["atr"]),
23         quote_to_usd_rate=quote_to_usd,
24     )

```

Cod 3.2. Sinteza a fluxului de decizie din backtesting

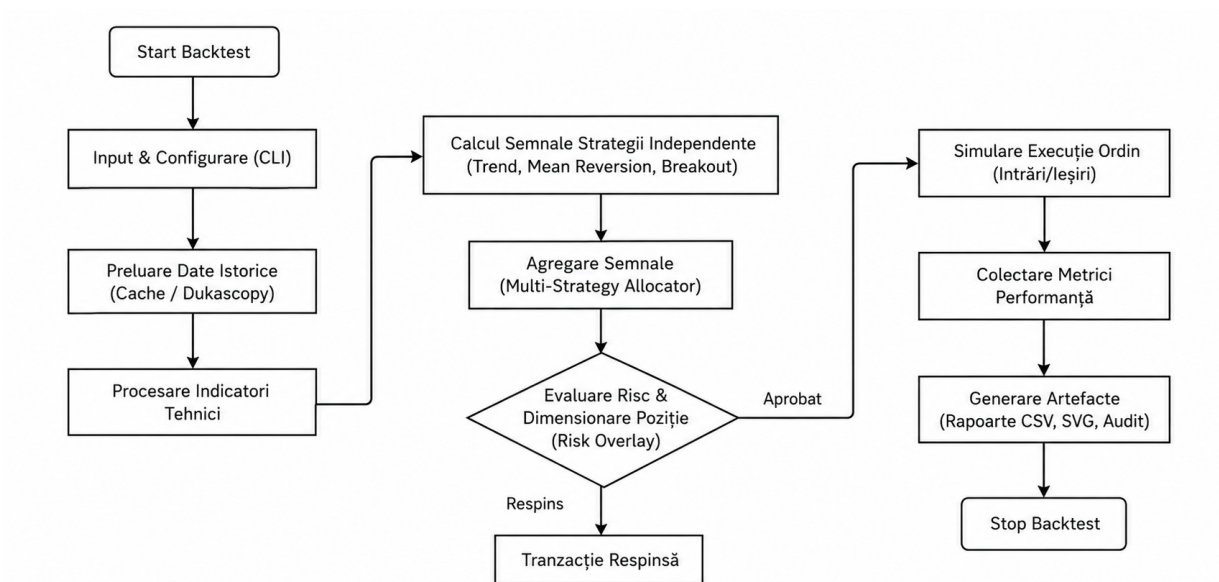


Figura 3.3. Fluxul de backtesting determinist

3.6. Strategiile implementate

Sistemul include trei strategii complementare.

Strategia de trend folosește EMA rapidă și lentă, ADX, MACD și confirmare multi-timeframe. Ea produce semnal long în regim *trend_up*, când EMA rapidă este peste EMA lentă, ADX depășește pragul configurat, iar MACD confirmă direcția. Pentru semnal short sunt verificate condiții simetrice în regim *trend_down*.

Strategia de revenire la medie folosește Bollinger Bands, RSI și filtre de calitate a lumânării. Ea este activă doar în regim *range*, cu ADX scăzut și volatilitate controlată. Semnalele apar când prețul se află la extremele canalului statistic, iar RSI indică supravânzare sau supracumpărare.

Strategia de breakout folosește canale Donchian și ATR. Ea caută închideri peste maximum canalului sau sub minimum canalului anterior, dar acceptă tranzacții doar când volatilitatea este suficient de ridicată. Această strategie este potrivită pentru trenduri și pentru episoade de creștere bruscă

a volatilității.

Complementaritatea strategiilor este importantă deoarece piața Forex nu rămâne permanent într-un singur comportament. O strategie de trend poate avea rezultate slabe într-o piață laterală, în timp ce o strategie de revenire la medie poate produce semnale greșite în timpul unui breakout puternic. Prin urmare, sistemul nu alege manual o singură strategie pentru toate perioadele, ci permite allocatorului să acorde ponderi diferite în funcție de performanță și context.

Tabelul 3.5. Rezumatul strategiilor implementate

Strategie	Indicatori utilizați	Regim țintă	Rol în sistem
Trend	EMA, ADX, MACD, confirmare multi-timeframe	trend_up, trend_down	Urmărește mișcări direcționale persistente.
Mean reversion	Bollinger Bands, RSI, ADX, filtre de lumânare	range	Caută revenirea prețului de la extreme statistice.
Breakout	Donchian Channels, ATR, z-score volatilitate	trend sau volatility_spike	Caută străpungeri ale canalelor de preț.

3.7. Allocatorul de strategii

Fiecare strategie returnează un semnal, o încredere și metadate explicative. Allocatorul combină semnalele printr-un scor compus. Pentru fiecare strategie sunt evaluate patru componente:

- performanța recentă, calculată din win rate, profit factor, profit mediu și drawdown;
- încrederea declarată de strategie pentru semnalul curent;
- potrivirea cu regimul de piață;
- diversificarea față de istoricul semnalelor celorlalte strategii.

Scorul compus folosește ponderile 0,4 pentru performanță, 0,3 pentru încredere, 0,2 pentru potrivirea cu regimul și 0,1 pentru diversificare. Semnalul final este suma semnalelor individuale ponderate cu scorurile normalizate. Dacă valoarea finală depășește pragul configurat, bias-ul devine long; dacă scade sub pragul negativ, bias-ul devine short; în caz contrar sistemul rămâne flat.

3.8. Managementul riscului

Componenta RiskOverlay decide dacă un semnal poate fi executat. Prima verificare este starea circuit breaker-ului. Dacă drawdown-ul curent a depășit pragul configurat, tranzacționarea este dezactivată temporar. Apoi sunt validate prețul, ATR-ul, cursul de conversie către USD, nivelul de volatilitate și expunerea pe monede.

Dimensiunea poziției este calculată pornind de la riscul dorit:

$$size = \frac{equity \cdot risk_per_trade}{ATR \cdot atr_multiplier \cdot quote_to_usd} \quad (3.1)$$

Formula (3.1) este limitată suplimentar de expunerea maximă și de levierul noțional maxim. Stop-loss-ul este plasat la o distanță de ATR față de prețul de intrare, iar take-profit-ul este stabilit la o distanță mai mare, pentru a permite raporturi risc-recompensă favorabile. Pe parcursul tranzacției, sistemul poate muta stop-ul la break-even și poate activa trailing stop-ul.

Implementarea practică păstrează aceeași logică: se calculează suma maximă riscată, riscul pe unitate și dimensiunea permisă atât de risc, cât și de levierul noțional. Dimensiunea finală este minimul dintre cele două limite.

```

1 target_risk_amount = equity * self.config.risk_per_trade
2 stop_distance = atr_value * self.config.atr_stop_multiplier
3 usd_risk_per_unit = stop_distance * quote_to_usd_rate
4
5 size_by_risk = int(max(target_risk_amount / usd_risk_per_unit, 1))
6 max_notional_usd = equity * self.config.max_notional_leverage
7 size_by_notional = int(max(max_notional_usd / usd_notional_per_unit, 0))
8
9 size_units = min(size_by_risk, size_by_notional)

```

Cod 3.3. Dimensionarea poziției în funcție de risc și volatilitate

Regulile de risc nu sunt tratate ca o extensie a strategiilor, ci ca o componentă independentă. Această decizie de proiectare reduce riscul ca o strategie să își suprascrie propriile protecții și permite verificarea uniformă a tuturor semnalelor. De exemplu, indiferent dacă semnalul provine din trend sau din breakout, intrarea poate fi respinsă pentru volatilitate insuficientă, lichiditate anormală, expunere prea mare sau circuit breaker activ.

Tabelul 3.6. Motive posibile de respingere a unei intrări

Motiv	Semnificație
flat_signal	Allocatorul nu a produs un bias long sau short suficient de puternic.
invalid_atr_or_price	Prețul, ATR-ul sau cursul de conversie nu permit calculul riscului.
quiet_market_atr	Volatilitatea curentă este prea mică față de media recentă.
liquidity_range_spike	Bara curentă are o amplitudine anormală față de media ultimelor bare.
max_exposure	Deschiderea poziției ar depăși expunerea totală maximă permisă.
circuit_breaker	Drawdown-ul a depășit pragul configurat, iar tranzacționarea este oprită temporar.

3.9. Modul live paper trading

Modul live folosește Interactive Brokers prin biblioteca *ib_insync*. La pornire, sistemul se conectează la broker, încarcă date istorice pentru încălzirea indicatorilor și pornește fluxurile de bare și prețuri. Pentru fiecare bară completă, fluxul decizional este similar cu cel din backtest. Diferența principală este că intrările și ieșirile sunt trimise brokerului ca ordine de piață, iar prețul de execuție este preluat din răspunsul brokerului.

Sistemul include mecanisme operaționale importante: lock-uri locale pentru a preveni rula-rea mai multor procese pe aceeași pereche și același cont, heartbeat pentru conexiunea Interactive Brokers, detectarea lipsei de actualizări și reconectare automată. Aceste mecanisme nu garantează absența riscului operațional, dar cresc robustețea în mediu paper.

3.10. Observabilitate în timp real

Pentru modul live, proiectul include un mecanism de observabilitate bazat pe Prometheus și Grafana. Botul pornește local un server HTTP de metrici pe portul 18000, iar Prometheus colectează valorile la un interval de 5 secunde prin ținta `host.docker.internal:18000`. Grafana citește datele din Prometheus și prezintă dashboard-uri separate pentru EURUSD, GBPUSD și USDJPY.

Au fost definite metrici globale și metrici pe pereche valutară. Metricile globale urmăresc capitalul curent, profitul/pierderea realizată și numărul total de poziții deschise. Metricile etichetate

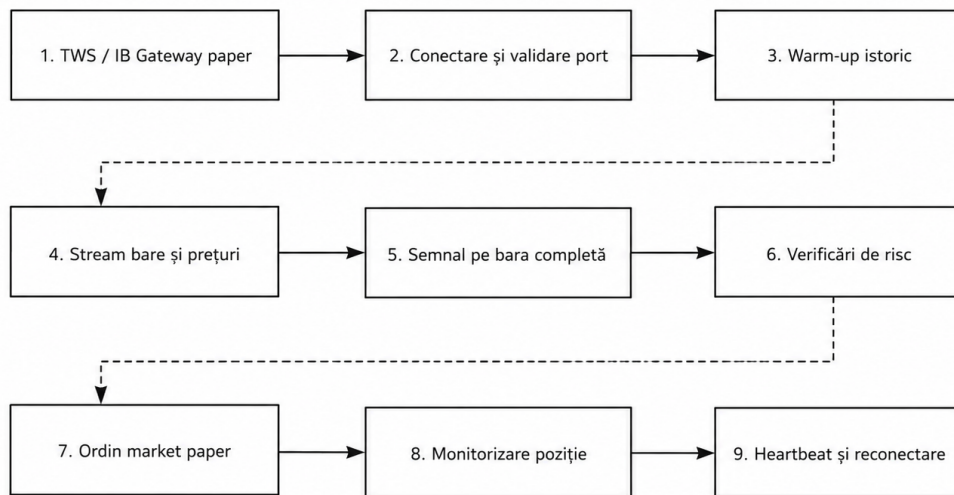


Figura 3.4. Fluxul de rulare live paper prin Interactive Brokers

cu pair urmăresc prețul live, semnalul allocatorului, poziția deschisă, dimensiunea poziției și P&L-ul realizat/nerealizat pentru fiecare paritate. Această separare permite analiza independentă a fiecărei perechi, fără amestecarea seriilor într-un singur grafic.

Serviciile de monitorizare rulează prin Docker Compose pe porturi non-default, pentru a evita conflictele cu alte aplicații locale: Grafana este expusă în browser la `localhost:33000`, Prometheus la `localhost:39090`, iar exporter-ul botului la `localhost:18000/metrics`. În interiorul rețelei Docker, Grafana accesează Prometheus prin adresa `http://prometheus:9090`, deoarece acesta este portul intern al containerului, nu portul publicat pe sistemul gazdă.

3.11. Artefacte generate

Fiecare rulare generează un director în output. Artefactele principale sunt:

- `trades.csv`: lista tranzacțiilor închise;
- `transactions.csv`: evenimente de intrare și ieșire;
- `equity_curve.csv`: evoluția capitalului;
- `strategy_metrics.csv`: performanța strategiilor;
- `regime_metrics.csv`: performanța pe regimuri de piață;
- `risk_overlay_metrics.csv`: deciziile componentei de risc;
- `summary.txt`: raport text cu metricile principale;
- `dashboard.html` și `performance_dashboard.svg`: vizualizări pentru analiză.

3.12. Algoritmul general

Algoritmul 3.1 Fluxul decizional al sistemului

```
1: procedure RUNBOT(config)
2:   Încarcă datele istorice sau fluxul live
3:   Calculează indicatorii și regimul de piață
4:   for fiecare bară completă do
5:     Actualizează poziția deschisă și capitalul
6:     Verifică stop-loss, take-profit, break-even și trailing stop
7:     Evaluează strategiile active
8:     Agregă semnalele prin allocator
9:     Verifică aprobarea componentei de risc
10:    if riscul este aprobat și nu există poziție deschisă then
11:      Deschide poziție long sau short
12:    Înregistrează tranzacții, metrici și curba capitalului
13:  Generează rapoarte CSV, text și dashboard
```

Capitolul 4. Testarea aplicației și rezultate experimentale

Testarea aplicației a urmărit verificarea fluxului complet, de la instalarea dependențelor și încărcarea datelor, până la calculul indicatorilor, generarea semnalelor, aplicarea regulilor de risc, înregistrarea tranzacțiilor și raportarea rezultatelor. Deoarece proiectul aparține unui domeniu financiar, evaluarea nu se limitează la funcționarea tehnică a codului. Rezultatele trebuie interpretate împreună cu riscurile, limitările și condițiile în care au fost obținute.

4.1. Mediul de testare

Aplicația este dezvoltată în Python 3 și folosește biblioteci pentru analiză de date, integrare cu brokerul, descărcare de date istorice și observabilitate live. Instalarea dependențelor se realizează prin:

```
1 pip install -r requirements.txt
```

Cod 4.1. Instalarea dependențelor proiectului

Pentru rulările experimentale au fost folosite date istorice Forex păstrate local în `data_cache`. Perechile analizate sunt EURUSD, GBPUSD și USDJPY, pe timeframe zilnic, în intervalul 2018-01-01 – 2024-12-31. Capitalul inițial implicit este 10.000 USD, iar riscul per tranzacție este 1%. Alegerea acestor perechi este justificată de faptul că sunt instrumente valutare majore, lichide și frecvent utilizate în testarea strategiilor Forex.

4.2. Punerea în funcțiune și configurarea aplicației

Punerea în funcțiune presupune pregătirea mediului Python, instalarea dependențelor și alegerea parametrilor de rulare. Fișierul `requirements.txt` definește bibliotecile necesare, iar fișierele din directorul `config` permit setarea perechilor și a parametrilor de strategie. Pentru backtesting, aplicația nu necesită conexiune la Interactive Brokers, deoarece datele sunt citite din cache local sau sunt descărcate prin furnizorul istoric configurat.

Parametrii importanți pentru o rulare sunt:

- `-mode`: selectează modul `backtest` sau `live`;
- `-pair` sau `-pairs`: definește perechea sau lista de perechi valutare;
- `-timeframe`: stabilește granularitatea barelor;
- `-start` și `-end`: definesc intervalul istoric pentru backtesting;
- `-initial-capital`: configurează capitalul inițial;
- `-risk-per-trade`: configurează riscul procentual pe tranzacție;
- `-slippage-bps`: permite simularea slippage-ului;
- `-ib-port`: definește portul Interactive Brokers pentru modul `live paper`.

Această abordare face ca experimentele să fie ușor de reprodus. O comandă completă descrie explicit perechea, intervalul, timeframe-ul, riscul și directorul de ieșire, iar rezultatele sunt salvate separat pentru fiecare rulare. Astfel, fiecare experiment poate fi reluat și verificat în aceleași condiții, aspect esențial pentru o evaluare tehnică riguroasă.

4.3. Scenarii de test

Au fost analizate trei scenarii principale:

- EURUSD pe timeframe zilnic, pentru o pereche majoră cu volatilitate moderată;
- GBPUSD pe timeframe zilnic, pentru o pereche cu mișcări mai ample și sensibilitate ridicată la evenimente macroeconomice;
- USDJPY pe timeframe zilnic, pentru o pereche cu comportament direcțional puternic în anumite perioade ale intervalului analizat.

Comenzile de rulare urmează aceeași structură:

```
1 python main.py --mode backtest --pair USDJPY --start 2018-01-01 \  
2   --end 2024-12-31 --timeframe D
```

Cod 4.2. Exemplu de rulare experimentală

4.4. Metrice urmărite

Pentru evaluarea rezultatelor au fost urmărite următoarele metrice:

- *Total P&L*: profitul sau pierderea netă în USD și procentual față de capitalul inițial;
- *max equity drawdown*: scăderea maximă a capitalului față de un maxim anterior;
- *profit factor*: raportul dintre profitul brut și pierderea brută;
- *procentul tranzacțiilor profitabile*: ponderea tranzacțiilor cu P&L pozitiv;
- *Sharpe și Sortino*: indicatori de performanță ajustată la risc;
- comparația cu strategia buy-and-hold pe aceeași pereche.

4.5. Metodologia de testare

Testarea a fost realizată prin rulări de backtesting pe aceleași condiții generale, pentru a putea compara rezultatele între perechi. Fiecare scenariu a folosit același interval istoric, același capital inițial și aceeași logică de risc. Diferențele rezultate provin din comportamentul perechii valutare și din modul în care strategiile reacționează la regimurile de piață identificate.

Pentru fiecare rulare au fost verificate următoarele aspecte:

- existența datelor istorice și ordonarea lor cronologică;
- generarea coloanelor de indicatori fără valori invalide în zona de tranzacționare;
- existența fișierelor de ieșire în directorul output;
- corelarea tranzacțiilor din `trades.csv` cu evenimentele din `transactions.csv`;
- calcularea sumarului text și a dashboard-ului vizual;
- respectarea limitelor de risc declarate.

Testarea modului live a fost tratată separat, deoarece depinde de TWS sau IB Gateway și de disponibilitatea conexiunii Interactive Brokers. Pentru această componentă, accentul a fost pus pe protecțiile operaționale: porturi paper, lock-uri locale, heartbeat și reconectare.

Pereche	P&L	Drawdown	Tranzacții	Win rate	Profit factor
EURUSD	+389,81 USD	3,81%	21	52,38%	1,522
GBPUSD	-325,11 USD	5,16%	17	58,82%	0,533
USDJPY	+3.106,55 USD	3,19%	36	69,44%	3,743

Tabelul 4.1. Rezultate backtesting pe timeframe zilnic, 2018–2024

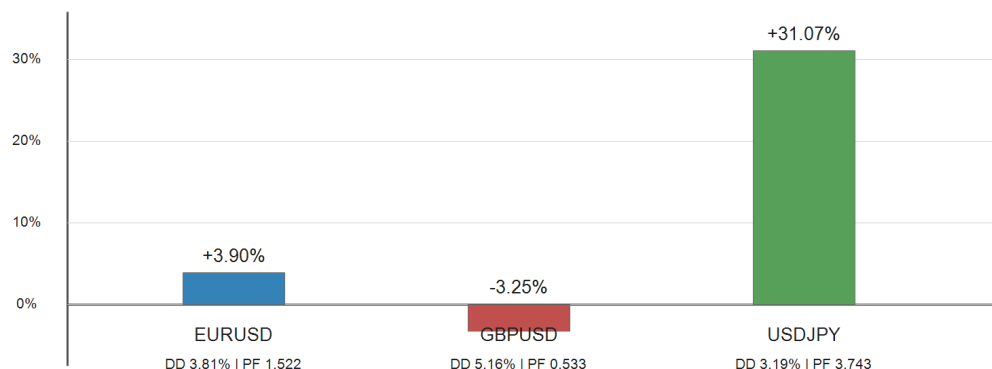


Figura 4.1. Comparație vizuală a rezultatelor de backtesting

4.6. Rezultate obținute

Rezultatele generate de rapoartele `summary.txt` sunt sintetizate în tabelul 4.1.

Rezultatele arată diferențe importante între perechi. Pentru EURUSD, sistemul a obținut un profit moderat de 3,90%, cu un drawdown de 3,81% și profit factor peste 1. Strategia a depășit semnificativ buy-and-hold pe intervalul analizat, deoarece buy-and-hold a avut rezultat negativ.

Pentru GBPUSD, deși rata tranzacțiilor profitabile a fost de 58,82%, profit factor-ul a fost subunitar, iar rezultatul final a fost negativ. Această situație arată că numărul tranzacțiilor câștigătoare nu este suficient; raportul dintre câștigul mediu și pierderea medie este critic. În acest caz, pierderile medii au fost mai mari decât profiturile medii.

Pentru USDJPY, sistemul a obținut cel mai bun rezultat: +31,07%, profit factor 3,743 și drawdown maxim de 3,19%. Strategia a depășit ușor buy-and-hold, ceea ce sugerează că mecanismele de intrare și ieșire au adăugat valoare într-un context direcțional favorabil.

4.7. Analiza riscului

Componenta de risc a limitat pierderile individuale în jurul pragului de 1% din capitalul disponibil, așa cum se observă în tranzacțiile închise prin *ATR Exit*. În același timp, mecanismele de break-even și trailing stop au permis protejarea profiturilor atunci când prețul s-a deplasat favorabil. Această combinație este vizibilă mai ales în rezultatele USDJPY, unde mai multe tranzacții au fost închise prin trailing stop sau take-profit ATR.

Drawdown-ul maxim a rămas sub 6% în toate cele trei scenarii analizate. Acest rezultat indică faptul că dimensionarea pozițiilor și limitarea expunerii au funcționat conform obiectivelor. Totuși, rezultatul negativ pe GBPUSD arată că protecția riscului nu transformă automat o strategie nepotrivită într-una profitabilă; ea controlează pierderea, dar selecția semnalelor rămâne decisivă.

4.8. Resurse, fiabilitate și scalabilitate

Aplicația este proiectată pentru rulări locale pe volume moderate de date OHLCV. Pentru scenariile analizate, datele zilnice pentru perioada 2018–2024 sunt reduse ca dimensiune și pot fi

procesate rapid pe un calculator obișnuit. În cazul timeframe-urilor mici, precum 1 minut sau 5 minute, numărul de bare crește semnificativ, iar timpul de rulare depinde de dimensiunea fișierelor CSV, de numărul strategiilor și de numărul perechilor analizate.

Din punct de vedere al memoriei, proiectul folosește `pandas DataFrame` pentru păstrarea datelor istorice și a indicatorilor. Această alegere este potrivită pentru claritate și prototipare, dar pentru volume foarte mari ar putea necesita optimizări, cum ar fi procesarea pe ferestre, reducerea coloanelor intermediare sau folosirea unui format de stocare mai eficient decât CSV. În stadiul actual, soluția este suficientă pentru backtesting educațional și pentru validarea logicii de tranzacționare pe perechi majore.

Fiabilitatea este susținută prin cache local, procesare deterministă și separarea artefactelor pe directoare de rulare. Dacă o rulare produce rezultate neașteptate, fișierele generate permit revenirea la tranzacții, evenimente, curba capitalului și motivele deciziilor de risc. În modul live, fiabilitatea operațională este îmbunătățită prin heartbeat, detectarea lipsei de actualizări și reconectare automată.

Scalabilitatea este limitată în principal de faptul că backtest-ul procesează secvențial fiecare bară și fiecare strategie. Pentru extindere, se pot rula perechi diferite în procese separate, se pot paraleliza experimentele sau se poate introduce un mecanism de optimizare distribuită. Aceste extinderi nu au fost incluse deoarece obiectivul proiectului este claritatea arhitecturală, nu execuția pe infrastructură de producție.

4.9. Aspecte de securitate și siguranță operațională

Într-un sistem de tranzacționare, securitatea nu se referă doar la autentificare, ci și la prevenirea utilizării greșite a aplicației. Cea mai importantă măsură din proiect este limitarea conexiunii Interactive Brokers la porturile paper trading 7497 și 4002. Dacă utilizatorul încearcă să conecteze aplicația la un port nepermis, brokerul refuză conexiunea. Această decizie reduce riscul trimiterii accidentale de ordine către un cont real.

Aplicația nu stochează parole sau chei API în codul sursă. Conexiunea cu Interactive Brokers se realizează prin sesiunea locală TWS sau IB Gateway, iar controlul accesului rămâne la nivelul platformei brokerului. În plus, lock-urile locale previn rularea simultană a două procese care ar putea administra aceeași pereche și același cont paper, reducând riscul de conflicte operaționale.

Totuși, aceste mecanisme nu transformă aplicația într-un sistem de producție complet securizat. Pentru utilizare reală ar fi necesare audit de cod, jurnalizare persistentă mai detaliată, management al erorilor la nivel de infrastructură, monitorizare externă și proceduri clare de oprire de urgență.

4.10. Testarea modului live

Modul live a fost proiectat pentru paper trading prin Interactive Brokers. Pentru utilizare, sunt necesari următorii pași:

- pornirea TWS sau IB Gateway în cont paper;
- activarea accesului API;
- folosirea portului 7497 pentru TWS paper sau 4002 pentru Gateway paper;
- rularea aplicației cu `-mode live`.

O comandă tipică este:

```
1 python main.py --mode live --pairs EURUSD,GBPUSD,USDJPY --timeframe 15m \  
2 --ib-port 7497 --client-id 14
```

Cod 4.3. Exemplu de rulare în mod live paper

Testarea live trebuie realizată exclusiv pe cont paper. Sistemul include protecții software, însă acestea nu elimină riscul operațional generat de configurări greșite, întreruperi de conexiune sau comportamente neprevăzute ale brokerului.

4.11. Monitorizare live cu Prometheus și Grafana

Pentru validarea comportamentului live a fost configurat un stack local de observabilitate format din Prometheus și Grafana. Botul rulează nativ pe Windows și expune metricile la adresa `localhost:18000/metrics`. Prometheus rulează într-un container Docker și accesează botul prin `host.docker.internal:18000`, iar Grafana se conectează la Prometheus prin adresa internă Docker `http://prometheus:9090`.

Porturile publicate pe sistemul gazdă au fost schimbate față de valorile implicite pentru a evita conflictele cu alte aplicații locale. Astfel, Prometheus este accesibil în browser la <http://localhost:39090>, iar Grafana la <http://localhost:33000>. Verificarea minimă a monitorizării constă în:

- verificarea endpoint-ului <http://localhost:18000/metrics>;
- confirmarea metricii `bot_live_price`;
- confirmarea metricilor per pereche pentru semnal și dimensiunea poziției;
- verificarea paginii <http://localhost:39090/targets>;
- confirmarea stării UP pentru ținta Prometheus;
- importarea dashboard-urilor Grafana din directorul `grafana/dashboards`;
- verificarea dashboard-urilor separate pentru EURUSD, GBPUSD și USDJPY.

Panourile Grafana afișează prețul live, semnalul allocatorului, starea poziției, dimensiunea poziției și P&L-ul perechii. Când botul nu deschide poziții, valorile pentru P&L și dimensiunea poziției pot rămâne la zero. Acest comportament indică starea *Flat*, nu o problemă a monitorizării.

4.12. Capturi de ecran din testare și monitorizare

Capturile de ecran din această secțiune completează rezultatele numerice prezentate anterior și documentează modul în care aplicația a fost verificată practic. Primele două figuri surprind dashboard-ul HTML generat după rularea de backtesting, iar următoarele ilustrează monitorizarea live și confirmarea execuțiilor în mediul paper trading al Interactive Brokers. În capturile finale se urmărește folosirea unei teme deschise și decuparea strictă pe zona relevantă, astfel încât textul și graficele să rămână lizibile în format tipărit.

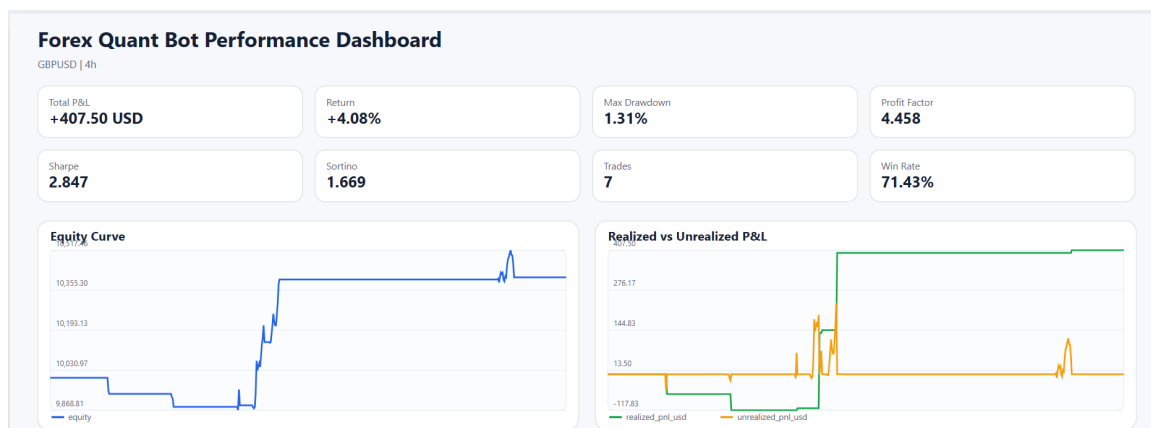


Figura 4.2. Dashboard-ul HTML generat după rularea de backtesting

Dashboard-ul de backtesting oferă o imagine sintetică asupra performanței: profitul total, randamentul, drawdown-ul maxim, factorul de profit, numărul tranzacțiilor și rata de câștig. În aceeași interfață sunt reprezentate evoluția capitalului și comparația dintre profitul realizat și profitul nerealizat, ceea ce permite verificarea rapidă a comportamentului strategiei pe intervalul analizat.

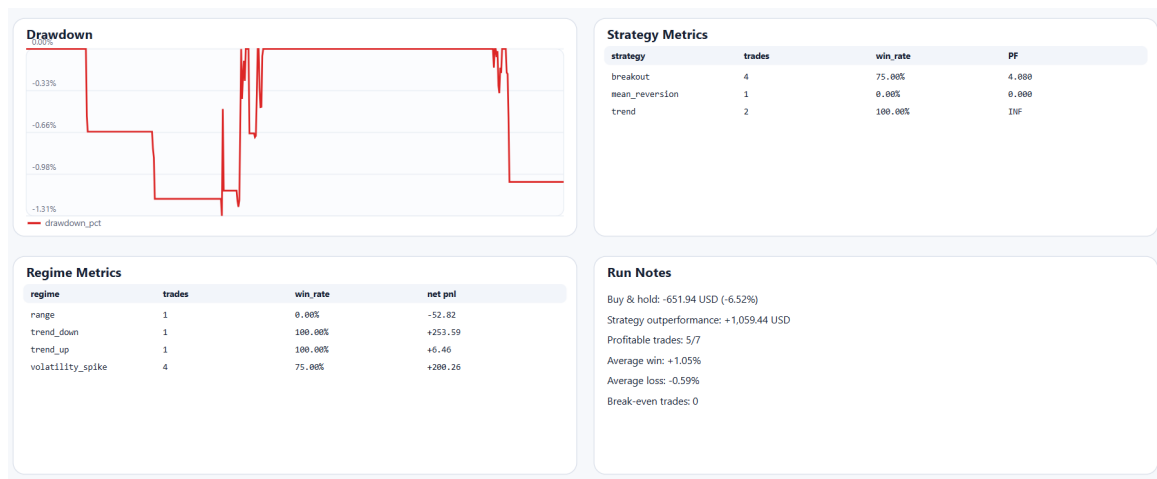


Figura 4.3. Metrice detaliate pentru drawdown, strategii și regimuri de piață

A doua captură evidențiază zonele de analiză detaliată ale dashboard-ului: evoluția drawdown-ului, performanța pe strategii, performanța pe regimuri de piață și notele generate pentru rularea respectivă. Aceste informații sunt utile pentru interpretarea rezultatelor, deoarece nu se limitează la profitul final, ci arată și contextul în care tranzacțiile au fost produse.

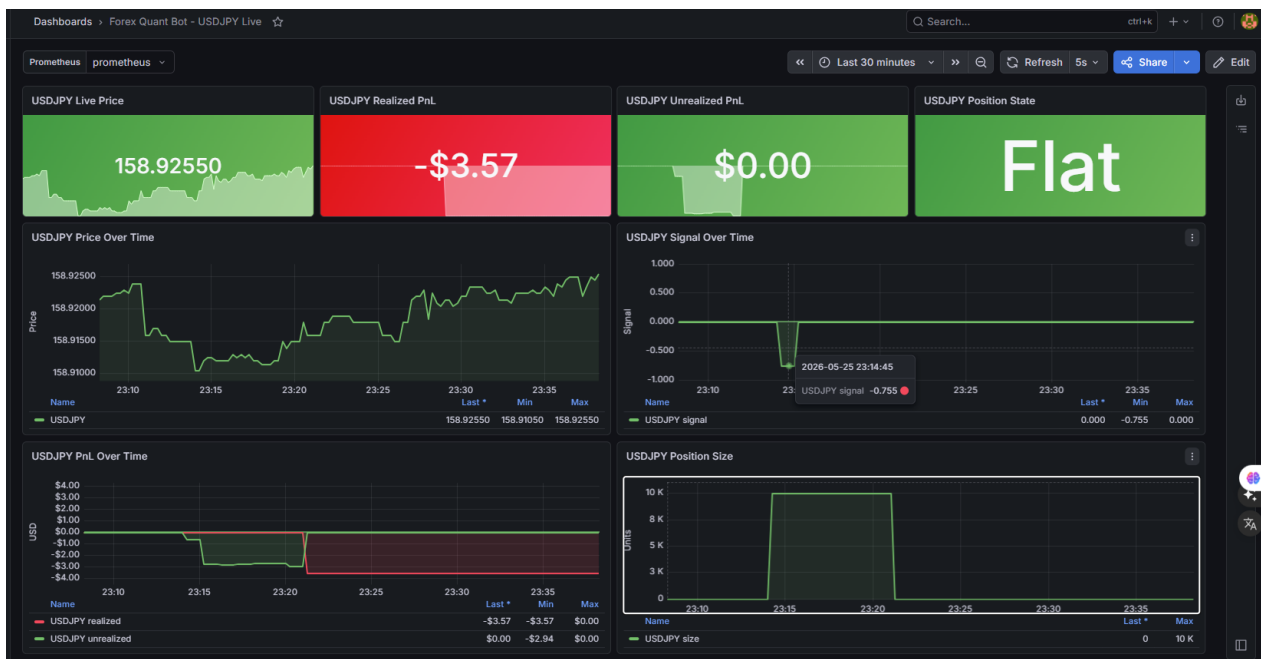


Figura 4.4. Dashboard Grafana pentru monitorizarea perechii USDJPY în regim live paper

Pentru rularea live paper, dashboard-ul Grafana afișează prețul curent, profitul realizat, profitul nerealizat, starea poziției, semnalul allocatorului și dimensiunea poziției. În exemplul din figura 4.4, perechea USDJPY este monitorizată separat, ceea ce confirmă că observabilitatea poate fi urmărită individual pentru fiecare instrument tranzacționat.



Figura 4.5. Notificare de ordin executat în Interactive Brokers paper trading

ACTIVITY	Live	Trades	Summary	+					
+/-	Time	Fin Instrument	Action	Quantity	Price	Exch.	Account Comi		
	23:21:01	USD.JPY	BOT	10K	158.925	IDEALPRO	DUK853859		
	23:14:02	USD.JPY	SLD	10K	158.900	IDEALPRO	DUK853859		

Figura 4.6. Înregistrarea tranzacțiilor în interfața Interactive Brokers

Capturile din Interactive Brokers confirmă faptul că semnalele produse de bot pot fi transformate în ordine paper și că execuțiile apar în interfața platformei. Notificarea de ordin executat și lista tranzacțiilor oferă o verificare vizuală a integrării dintre aplicația Python și mediul de tranzacționare.

4.13. Limitări

Rezultatele experimentale depind de intervalul istoric, timeframe și parametrii configurați. Nu au fost incluse comisioane reale, spread variabil și latență de execuție complet realistă. De asemenea, backtesting-ul pe o singură perioadă nu este suficient pentru validare finală; sunt necesare teste out-of-sample, walk-forward și verificări suplimentare pe condiții de piață diferite.

Capitolul 5. Concluzii

Lucrarea a avut ca obiectiv proiectarea și implementarea unui sistem de tranzacționare algoritmică pentru piața Forex, cu accent pe testarea strategiilor prin backtesting, pe controlul riscului și pe validarea operațională în regim paper trading. Pe parcursul dezvoltării, accentul nu a fost pus doar pe obținerea unor rezultate numerice, ci și pe construirea unui flux coerent, reproductibil și ușor de analizat.

Sistemul implementat îndeplinește obiectivele formulate în introducere. Aplicația permite rularea de backtest-uri deterministe pe date istorice, suportă mai multe perechi valutare și timeframe-uri, integrează strategii adaptate unor regimuri de piață diferite și generează artefacte utile pentru interpretarea rezultatelor. În plus, integrarea cu Interactive Brokers este limitată explicit la paper trading, ceea ce permite verificarea comportamentului live fără expunere financiară reală.

O contribuție importantă a proiectului este separarea clară dintre semnalul strategiei și decizia de risc. Această separare reflectă o idee esențială în tranzacționarea algoritmică: un semnal tehnic nu este suficient pentru deschiderea unei poziții. Tranzacția trebuie validată și din perspectiva volatilității, a expunerii totale, a dimensiunii poziției și a drawdown-ului. Prin urmare, sistemul nu tratează managementul riscului ca pe o extensie minoră a strategiei, ci ca pe o componentă independentă a arhitecturii.

Rezultatele experimentale au arătat că performanța diferă semnificativ între perechi valutare. USDJPY a produs cel mai bun rezultat în perioada analizată, în timp ce GBPUSD a avut rezultat negativ, chiar dacă rata tranzacțiilor câștigătoare a fost peste 50%. Această observație evidențiază faptul că evaluarea unei strategii nu trebuie redusă la procentul de tranzacții profitabile. Sunt la fel de importante profit factor-ul, drawdown-ul, raportul dintre câștigul mediu și pierderea medie, precum și stabilitatea comportamentului pe mai multe instrumente.

5.1. Contribuții ale lucrării

Principalele contribuții realizate în cadrul lucrării sunt:

- proiectarea unei arhitecturi modulare pentru backtesting și paper trading pe perechi Forex (detaliată în Secțiunea 3.3);
- implementarea unui flux determinist de procesare bară cu bară, fără acces la informații viitoare (prezentat în Secțiunea 3.5);
- integrarea a trei strategii complementare, adaptate pentru trend, revenire la medie și breakout (descrise în Secțiunea 3.6);
- realizarea unui allocator multi-strategie care utilizează performanța recentă, încrederea semnalului, regimul de piață și diversificarea (detaliat în Secțiunea 3.7);
- implementarea unei componente de risc independente, cu dimensionarea pozițiilor pe baza ATR, limitarea expunerii și mecanism de tip circuit breaker (detaliată în Secțiunea 3.8);
- generarea de artefacte pentru auditarea tranzacțiilor, evenimentelor, riscului și performanței (prezentată în Secțiunea 3.11);
- integrarea controlată cu Interactive Brokers în regim paper trading (descrisă în Secțiunea 3.9);
- adăugarea unui mecanism de observabilitate live prin Prometheus și Grafana, cu dashboard-uri separate pentru perechile valutare monitorizate (demonstrat în Secțiunea 3.10).

Prin aceste contribuții, lucrarea depășește nivelul unei strategii izolate și propune un cadru complet de evaluare și execuție controlată. Valoarea principală a proiectului constă în transparența fluxului decizional: fiecare rezultat poate fi urmărit înapoi către datele folosite, strategia care a generat semnalul, decizia allocatorului și regulile de risc aplicate.

5.2. Comparatie cu soluții similare

Comparativ cu platformele comerciale de tranzacționare algoritmică, soluția propusă este mai restrânsă ca funcționalitate, dar oferă un grad mai mare de transparență asupra implementării. Codul sursă este organizat astfel încât fiecare componentă să aibă un rol clar, iar rezultatele sunt exportate în formate simple, ușor de verificat. Această abordare este potrivită pentru un proiect de licență, deoarece pune accent pe înțelegerea mecanismelor și pe justificarea deciziilor tehnice.

În același timp, proiectul are limitări. Sistemul nu include încă o metodologie completă de validare walk-forward, nu modelează în detaliu spread-ul variabil și slippage-ul, iar rezultatele obținute prin backtesting trebuie interpretate cu prudență. De asemenea, integrarea live este destinată exclusiv mediului paper trading și nu trebuie considerată pregătită pentru utilizare pe cont real fără audit, monitorizare suplimentară și mecanisme operaționale mai robuste.

5.3. Direcții viitoare de dezvoltare

Direcțiile viitoare de dezvoltare includ:

- adăugarea unor teste automate unitare și de integrare pentru componentele critice;
- implementarea unei metodologii walk-forward, pentru reducerea riscului de supraoptimizare;
- includerea spread-ului variabil, a comisioanelor și a slippage-ului estimat din date reale;
- extinderea dashboard-urilor cu alerte, comparații între rulări și rapoarte periodice;
- optimizarea parametrilor per pereche valutară, cu separare clară între datele de antrenare și cele de validare;
- adăugarea unor strategii suplimentare sau a unor modele statistice mai avansate;
- îmbunătățirea monitorizării live prin jurnale persistente, alerte și proceduri de oprire controlată.

În ansamblu, proiectul oferă o bază funcțională pentru studierea tranzacționării algoritmice Forex și pentru dezvoltări ulterioare orientate către evaluare mai robustă, monitorizare live și control mai avansat al riscului. Lucrarea demonstrează că un sistem de tranzacționare algoritmică nu trebuie privit doar ca un generator de semnale, ci ca un ansamblu de componente care trebuie să colaboreze într-un mod verificabil, transparent și responsabil.

Bibliografie

- [1] E. P. Chan, *Algorithmic Trading: Winning Strategies and Their Rationale*. John Wiley & Sons, 2013, ISBN: 978-1-118-46014-6. [Online]. Available: <https://www.wiley.com/en-us/Algorithmic+Trading%3A+Winning+Strategies+and+Their+Rationale-p-9781118460146>
- [2] D. H. Bailey, J. M. Borwein, M. L. de Prado, and Q. J. Zhu, “The probability of backtest overfitting,” *Journal of Computational Finance*, vol. 20, no. 4, pp. 39–69, 2017, DOI: 10.21314/JCF.2016.322. [Online]. Available: <https://doi.org/10.21314/JCF.2016.322>
- [3] The pandas development team, “pandas documentation,” <https://pandas.pydata.org/docs/>, 2026.
- [4] NumPy Developers, “NumPy documentation,” <https://numpy.org/doc/>, 2026.
- [5] Dukascopy Python Contributors, “dukascopy-python package documentation,” <https://pypi.org/project/dukascopy-python/>, 2026.
- [6] E. de Wit, “ib_insync: Python API for Interactive Brokers,” https://github.com/erdewit/ib_insync, 2024.
- [7] Interactive Brokers, “Interactive Brokers API Documentation,” <https://www.interactivebrokers.com/campus/ibkr-api-page/twsapi-doc/#api-introduction>, 2026.
- [8] Prometheus Authors, “Prometheus documentation,” <https://prometheus.io/docs/introduction/overview/>, 2026.
- [9] Grafana Labs, “Grafana documentation,” <https://grafana.com/docs/grafana/latest/>, 2026.
- [10] Docker Inc., “Docker Compose documentation,” <https://docs.docker.com/compose/>, 2026.

Anexe

Anexa 1. Comenzi de rulare utilizate

Această anexă prezintă comenzile principale folosite pentru instalarea și testarea aplicației.

```
1 pip install -r requirements.txt
2
3 python main.py --mode backtest --pair EURUSD --start 2018-01-01 \
4   --end 2024-12-31 --timeframe D
5
6 python main.py --mode backtest --pair GBPUSD --start 2018-01-01 \
7   --end 2024-12-31 --timeframe D
8
9 python main.py --mode backtest --pair USDJPY --start 2018-01-01 \
10  --end 2024-12-31 --timeframe D
```

Cod 1. Comenzi pentru instalare și backtesting

Pentru modul live paper trading, aplicația se pornește după ce TWS sau IB Gateway rulează în cont paper și are activat accesul API:

```
1 python main.py --mode live --pairs EURUSD,GBPUSD,USDJPY --timeframe 15m \
2   --ib-port 7497 --client-id 14
```

Cod 2. Comandă pentru rularea în mod live paper

Pentru monitorizarea live se pornesc serviciile Prometheus și Grafana prin Docker Compose:

```
1 docker compose up -d
```

Cod 3. Pornirea serviciilor de observabilitate

Adresele utilizate pentru verificare sunt:

- `http://localhost:18000/metrics` pentru metricile expuse de bot;
- `http://localhost:39090/targets` pentru starea țintei Prometheus;
- `http://localhost:33000` pentru interfața Grafana.

Anexa 2. Metrici de observabilitate live

Pentru monitorizarea modului live paper, aplicația expune metrici Prometheus care descriu starea curentă a botului, prețurile urmărite și pozițiile asociate perechilor valutare. Aceste metrici sunt utile atât pentru verificarea rapidă a funcționării sistemului, cât și pentru construirea dashboard-urilor Grafana.

Metrică	Rol
bot_live_price{pair}	Prețul live pentru fiecare pereche valutară monitorizată.
bot_pair_signal{pair}	Semnalul numeric generat de allocator pentru perechea respectivă.
bot_pair_open_position{pair}	Indicator binar care arată dacă perechea are o poziție deschisă.
bot_pair_position_size_units{pair}	Dimensiunea poziției, exprimată în unități ale instrumentului tranzacționat.
bot_pair_realized_pnl_usd{pair}	Profitul sau pierderea realizată pentru perechea valutară.
bot_pair_unrealized_pnl_usd{pair}	Profitul sau pierderea nerealizată pentru poziția curentă.
bot_equity_usd	Capitalul curent raportat de aplicație.
bot_open_positions	Numărul total al pozițiilor deschise.

Tabelul A.1. Metrici Prometheus expuse de aplicația live

Exemple de interogări Prometheus utile pentru verificare sunt:

```

1 up{job="forex_quant_bot"}
2 bot_live_price{pair="EURUSD"}
3 bot_pair_signal{pair="USDJPY"}
4 bot_pair_position_size_units{pair="USDJPY"}
5 bot_pair_realized_pnl_usd{pair="USDJPY"}

```

Cod 4. Interogări Prometheus pentru monitorizarea botului

Anexa 3. Procedură de verificare pentru rularea live paper

Înainte de pornirea modului live paper, este necesară verificarea mediului de execuție și a conexiunilor externe. Procedura recomandată este:

1. pornirea TWS sau IB Gateway în cont paper;
2. verificarea portului API, de regulă 7497 pentru TWS paper sau 4002 pentru IB Gateway paper;
3. pornirea botului cu perechile valutare dorite și cu un client-id unic;
4. verificarea endpoint-ului /metrics în browser;
5. verificarea stării UP în pagina /targets din Prometheus;
6. deschiderea dashboard-ului Grafana corespunzător perechii valutare monitorizate;
7. verificarea în Interactive Brokers a ordinelor paper generate și a pozițiilor curente.

Această procedură reduce riscul de configurare greșită și permite confirmarea faptului că botul, brokerul paper și sistemul de observabilitate funcționează împreună.

Anexa 4. Structura artefactelor generate

Fiecare rulare creează un director separat în output. Numele directorului include modul de rulare, perechea valutară, timeframe-ul, perioada testată și timestamp-ul rulării.

Fișier	Rol
trades.csv	Tranzacțiile închise, cu prețuri de intrare/ieșire, P&L, regim și motivul semnalului.
transactions.csv	Evenimentele individuale de intrare și ieșire pentru fiecare tranzacție.
equity_curve.csv	Evoluția capitalului după fiecare bară procesată.
returns.csv	Tabel sumar pentru profit brut, pierdere brută, profit factor și comisioane.
strategy_metrics.csv	Metrici asociate performanței strategiilor.
regime_metrics.csv	Performanța grupată după regimul de piață al intrării.
risk_overlay_metrics.csv	Deciziile componente de risc și motivele aprobării sau respingerii semnalelor.
summary.txt	Raport text cu indicatorii principali și lista tranzacțiilor.
dashboard.html	Dashboard HTML pentru inspectarea vizuală a performanței.

Tabelul A.2. Fișiere generate de o rulare de backtesting

Anexa 5. Parametri relevanți de configurare

Aplicația folosește clase de configurare pentru strategii, risc, date, logare și broker. Parametrii principali ai componente de risc sunt:

Parametru	Valoare implicită	Descriere
initial_capital	10.000 USD	Capitalul inițial al simulării.
risk_per_trade	0,01	Ponderea capitalului riscată per tranzacție.
max_total_exposure	0,05	Expunerea totală maximă permisă.
drawdown_circuit_breaker	0,05	Pragul de drawdown care oprește temporar tranzacționarea.
atr_stop_multiplier	1,5	Multiplicatorul ATR pentru stop-loss.
take_profit_atr_multiplier	5,0	Multiplicatorul ATR pentru take-profit.
max_notional_leverage	1,0	Levierul notional maxim permis.

Tabelul A.3. Parametri de risc utilizați în proiect

Anexa 6. Fragmente relevante din codul sursă

Această anexă include fragmente reprezentative din implementare. Au fost selectate componente care susțin direct arhitectura descrisă în Capitolul 3: modelele interne de date, alocarea semnalelor, controlul riscului, protecțiile pentru rularea live și persistarea artefactelor.

6.1. Modele interne pentru decizie, risc și tranzacții

Modelele de tip `dataclass` definesc obiectele transmise între strategii, allocator, componenta de risc și modulele de raportare. Separarea acestor structuri ajută la auditarea deciziilor și la exportul coerent al rezultatelor.

```
1 @dataclass(slots=True)
2 class StrategyDecision:
3     name: str
4     signal: int
5     confidence: float
6     metadata: dict[str, Any] = field(default_factory=dict)
7
8
9 @dataclass(slots=True)
10 class CompositeDecision:
11     final_signal: float
12     bias: int
13     strategy_scores: dict[str, float]
14     strategy_decisions: dict[str, StrategyDecision]
15     metadata: dict[str, Any] = field(default_factory=dict)
16
17
18 @dataclass(slots=True)
19 class RiskDecision:
20     approved: bool
21     size_units: int
22     stop_distance: float
23     stop_price: float
24     risk_amount_usd: float
25     exposure_after_trade: float
26     reason: str
```

Cod 5. Modele interne pentru semnale compuse și decizii de risc

6.2. Calculul scorului în allocatorul de strategii

Fragmentul următor arată modul în care allocatorul combină performanța recentă, încrederea semnalului, potrivirea cu regimul de piață și diversificarea. Rezultatul este un semnal numeric final și un bias operațional: long, short sau flat.

```
1 for name, decision in decisions.items():
2     snapshot = performance_tracker.snapshot(name, current_regime)
3     performance_score = self._performance_score(snapshot)
4     raw_confidence = (
5         decision.confidence
6         if decision.confidence >= self.min_strategy_confidence
7         else 0.0
8     )
9     regime_fit = snapshot.regime_fit
10    diversification = self._diversification_bonus(name)
11    composite = (
12        0.4 * performance_score
13        + 0.3 * raw_confidence
14        + 0.2 * regime_fit
```

```

15         + 0.1 * diversification
16     )
17     if decision.signal == 0:
18         composite *= 0.25
19     raw_scores[name] = composite
20
21 scores = normalize_scores(raw_scores)
22 final_signal = float(
23     sum(decisions[name].signal * scores[name] for name in decisions)
24 )
25 if final_signal > self.score_threshold:
26     bias = 1
27 elif final_signal < -self.score_threshold:
28     bias = -1
29 else:
30     bias = 0

```

Cod 6. Agregarea scorurilor în allocatorul multi-strategie

6.3. Circuit breaker pentru controlul drawdown-ului

Componenta de risc urmărește capitalul maxim atins și dezactivează temporar tranzacționarea atunci când drawdown-ul depășește pragul configurat. Reluarea tranzacționării este permisă doar după terminarea perioadei de cooldown și după revenirea capitalului peste pragul de recuperare.

```

1 def update_equity(self, equity: float) -> None:
2     self.equity_peak = max(self.equity_peak, equity)
3     if self.equity_peak <= 0:
4         return
5
6     drawdown_pct = max(0.0, 1.0 - (equity / self.equity_peak))
7     recovery_ratio = equity / self.equity_peak if self.equity_peak else 0.0
8     breached_now = (
9         self.last_drawdown_pct
10        < self.config.drawdown_circuit_breaker
11        <= drawdown_pct
12    )
13
14    if not self.circuit_breaker_active and breached_now:
15        self.trading_enabled = False
16        self.circuit_breaker_active = True
17        self.cooldownBars_remaining = max(
18            self.config.circuit_breaker_cooldownBars, 0
19        )
20    elif self.circuit_breaker_active:
21        if self.cooldownBars_remaining > 0:
22            self.cooldownBars_remaining -= 1
23        recovery_ready = (
24            recovery_ratio >= self.config.circuit_breaker_recovery_threshold
25        )
26        cooldown_complete = self.cooldownBars_remaining <= 0
27        if cooldown_complete and recovery_ready:
28            self.trading_enabled = True
29            self.circuit_breaker_active = False
30            self.cooldownBars_remaining = 0
31
32    self.last_drawdown_pct = drawdown_pct

```

Cod 7. Mecanism de tip circuit breaker pe baza drawdown-ului

6.4. Protecție împotriva rulărilor live duplicate

În modul live, aplicația creează fișiere lock pentru perechile administrate. Acest mecanism reduce riscul ca două procese să gestioneze simultan aceeași pereche valutară și același cont paper.

```

1 def _acquire_live_pair_locks(self) -> bool:
2     if self.config.allow_duplicate_live_pair:
3         return True
4     lock_dir = self.config.logging.output_dir / ".live_locks"
5     lock_dir.mkdir(parents=True, exist_ok=True)
6     acquired: list[Path] = []
7
8     for pair in self.config.pairs:
9         lock_path = lock_dir / f"{self._live_lock_key(pair)}.lock"
10        payload = {
11            "pid": os.getpid(),
12            "pair": pair,
13            "timeframe": self.config.timeframe,
14            "host": self.config.ib.host,
15            "port": self.config.ib.port,
16            "account": self.config.ib.account or "default",
17            "client_id": self.config.ib.client_id,
18        }
19        if self._try_create_live_lock(lock_path, payload):
20            acquired.append(lock_path)
21            continue
22
23        existing = self._read_live_lock(lock_path)
24        existing_pid = int(existing.get("pid", 0) or 0) if existing else 0
25        if existing_pid and self._pid_exists(existing_pid):
26            self._release_live_pair_locks(acquired)
27            return False
28
29    self.live_pair_lock_paths.extend(acquired)
30    return True

```

Cod 8. Mecanism de lock pentru evitarea rulărilor live duplicate

6.5. Persistarea artefactelor de analiză

După fiecare rulare, rezultatele sunt salvate în fișiere separate. Această abordare permite verificarea tranzacțiilor, a deciziilor de risc, a curbei capitalului și a rapoartelor vizuale fără a depinde de o interfață grafică.

```

1 def persist_report(self, report: dict[str, Any], summary_text: str) -> dict[str, Path]:
2     paths: dict[str, Path] = {}
3     for key, filename, include_index in [
4         ("trade_records", "trades.csv", False),
5         ("transactions", "transactions.csv", False),
6         ("strategy_metrics", "strategy_metrics.csv", False),
7         ("regime_metrics", "regime_metrics.csv", False),
8         ("risk_metrics", "risk_overlay_metrics.csv", False),
9         ("equity_curve", "equity_curve.csv", False),
10        ("returns_table", "returns.csv", True),
11        ("details_table", "details.csv", True),
12        ("order_events", "order_events.csv", False),
13        ("open_positions", "open_positions.csv", False),
14    ]:
15        value = report.get(key)
16        if isinstance(value, pd.DataFrame):
17            path = self.run_dir / filename

```

```
18         value.to_csv(path, index=include_index)
19         paths[key] = path
20
21     paths["summary"] = self.write_text("summary.txt", summary_text)
22     dashboard_paths = PerformanceDashboardWriter(self.run_dir).persist(report)
23     paths.update(dashboard_paths)
24     return paths
```

Cod 9. Persistarea rapoartelor CSV, text și dashboard